The Last Portal
Student Assignment

Data Structures programming assignment

Puzzle edition v1.5.1 | C99 | Windows and Linux

Your task

Repair **six missing C functions** and correct **two faulty controller routines** so that Caitlyn can complete three missions and return Home. You will edit the supplied source, rebuild the application and use the restored behavior in the game. The graphics and game engine are already provided.

A note before you begin

I know this is not your usual programming assignment. It is not a course to finish, a certificate to download and a file to upload. You will have to invest time and effort. You may get stuck, trace the same code several times and discover that a small mistake can break an entire system. That is part of the work.

I assure you that giving this an honest try will be worth it. This is a toy game, but it is a real running application, and the code you repair makes something happen on the screen. When the undo finally works, the bot follows your queue, or Caitlyn blocks an attack because your logic is correct, that success belongs to you.

Give yourself a chance to feel that dopamine: the satisfaction of making something work because you understood it. The confidence from that small success can help you face the next, harder problem. You can choose to do this only for the marks, or use it to build a skill you can rely on. That confidence is the high I hope you take away from this assignment.

The three missions

Mission	Structure	What your repair enables
1 A memory of a mistake	History stack	Undo a trapped crate push and open EXIT.
2 A route to remember	Circular queue	Remember a random route and deliver the power cell.
3 Clash of Queue and Stack	Linked stack and queue	Counter the alien and fire the finishing move.

Work in `puzzle/student/stack.c`, `queue.c`, `attack_list.c` and `diagnostics.c`. This handout explains the supplied design, every repair contract, the gameplay and the final submission.

Using your assignment pack

Extract the entire **The Last Portal Assignment Pack v1.5.1** ZIP before opening a game or editing code. Keep the folders together in a writable location such as your Documents folder. Start with `START_HERE.txt`, then use this handout as your repair reference.

Item in the extracted pack	What it contains and how to use it
`Assignment_Handout.pdf` and `.docx`	The same assignment in PDF and editable Word formats. Read the repair rules and submission requirements.
`Reference/`	The completed game for Windows x64 and Linux x86-64, with its artwork, fonts, sounds and licenses. It contains no completed C source, headers or build libraries.
`Puzzle/`	Your assignment project: starter source, build/check scripts, bundled development libraries, resources and the initial unfinished executables. Make all code repairs here.

Play the completed reference first

Use the completed game to understand the goal, controls and restored behavior in each room. All eight functions are already working in this executable, but you still play the three missions yourself. A reference playthrough does not repair or complete your Puzzle assignment.

Windows: open `Reference` and double-click `PLAY_WINDOWS.bat`. From PowerShell opened in that folder, use:

```
.\PLAY_WINDOWS.bat
```

Linux: open a terminal in `Reference` and use the following command. It also works if ZIP extraction has removed the launcher executable bit:

```
sh play-linux.sh
```

These launchers only open the supplied completed binary; they do not build anything. No compiler is needed to play it. Both versions need a graphical desktop and OpenGL support. Use the matching 64-bit operating system; Windows and Linux executables are not interchangeable.

Then repair your own Puzzle copy

Open `Puzzle`. To inspect the first repair screen before editing, open `build/puzzle/last_portal.exe` on Windows. On Linux, open a terminal in `Puzzle` and run the commands below:

```
chmod +x build/puzzle/last_portal
./build/puzzle/last_portal
```

Edit only the four files under `Puzzle/puzzle/student/`. Close the Puzzle game, save and rebuild it after every source change using the build instructions in this handout. Every project-relative path later in the handout is relative to **Puzzle**, not the outer pack folder.

Keep Reference and Puzzle separate. Do not replace the Puzzle executable with the completed one. Reference progress is stored as `Reference/completed.save`; your Puzzle room checkpoint is `Puzzle/puzzle.save`. Neither save file contains your source repairs. Close the relevant game and delete only its save file if you want a fresh playthrough.

Use the Reference to observe how a repaired mechanic should behave, then implement it yourself in Puzzle. Submit your repaired student source and the evidence requested at the end of this handout. A screenshot of the completed Reference is not evidence of your repair.

Doing the work yourself

Do not use an LLM to generate or directly solve the assignment. This includes asking ChatGPT or another model to fill the six missing bodies, rewrite either faulty controller into a finished solution, or solve each repair for you. Copying such an answer defeats the purpose, even if the resulting program runs.

You can choose whether you are doing this only for marks or for the skill and confidence you gain. I cannot make that choice for you. A working file produced by somebody else may look complete, but it cannot give you the experience of understanding a failure, tracing the state and making your own correction work. The mark is temporary; the ability to approach the next problem is what you take with you.

Use the supplied comments, headers, F1 guide, compiler messages, debugger and C documentation. Ask for help understanding a concept or a tool when needed. General conceptual help from an LLM is different from handing it these repair functions and asking for their implementations: keep the actual diagnosis, implementation and testing your own. Do not paste generated repair code into your submission.

Submit a repair only when you can explain its state changes and boundary cases yourself. You should be able to trace a push/pop, a wrapped queue, a changed head pointer and a controller decision without relying on a generated explanation.

The repair cycle

1. Read the repair screen and press F1 if you need the room guide.

2. Close the game. Open the named student C file and replace the indicated starter body, or repair the faulty controller logic.

3. Save your source and rebuild using the platform instructions on the next page. Fix compiler errors before continuing.

4. Use the repaired mechanic in the game. The next check or mission reveals the next task. Repeat until Caitlyn reaches Home.

What the placeholder lines mean

Starter line	Meaning and required action
`repair_required(...)`	A supplied engine marker displays the missing-function panel. Remove the call when replacing that body. Do not implement this helper or disable its checks.
`(void)parameter;`	Suppresses an unused-parameter warning in a stub. Remove it when your implementation uses the parameter.
`return false;`	The stub always reports failure. Your finished operation returns true on success and false for its documented rejection cases.

All eight repair routines return `bool`. A successful operation is `true`; empty, full, not-found or invalid-input rejection is `false`. Use `NULL` for pointers, not as a replacement for the specified boolean contract.

Building after a source change

Yes, you must rebuild after editing C source. Saving a file does not change an existing executable. In the extracted assignment pack, open `Puzzle` and keep its subfolders together. Run all commands on this page inside `Puzzle`, where the build scripts are located, not inside the ZIP or the outer pack folder.

The release includes a starter executable: `build/puzzle/last_portal.exe` on Windows and `./build/puzzle/last_portal` on Linux. Opening it is enough to inspect the first repair screen. After changing source, use the commands below to build a new executable.

Windows using PowerShell

1. Save your edited C files and close the game. Open PowerShell in the extracted project folder. A 64-bit MinGW-w64 GCC compiler must be installed and available on PATH in that terminal. Use the compiler setup provided for your lab.

2. Check that PowerShell can find the compiler. The first command should print its version; the second should identify an x86-64 MinGW target.

```
gcc --version
gcc -dumpmachine
```

3. Compile and launch the Puzzle edition with this command. The leading dot and backslash are required for a local script in PowerShell.

```
.\run-windows.bat puzzle
```

4. If compilation succeeds, the script opens the new game. If it reports an error, read the first compiler message, correct your source, save and repeat. If gcc is not recognised, fix the compiler/PATH setup before attempting the build.

To compile without launching, use `.\build-windows.bat puzzle`. The rebuilt executable is `build/puzzle/last_portal.exe`. In Command Prompt, `run-windows.bat puzzle` also builds and runs it.

Linux using a terminal

1. Save your C files and close the game. Open a terminal in the extracted project folder. You need a C99 compiler, standard development tools, and the X11/OpenGL libraries required by the bundled raylib library. The game needs a graphical desktop.

2. Check the compiler, then build and launch. Use these commands from the project folder.

```
cc --version
./run-linux.sh puzzle
```

3. If a script reports Permission denied after extraction, make the supplied scripts executable, then run the command again.

```
chmod +x build-linux.sh run-linux.sh check-linux.sh
./run-linux.sh puzzle
```

4. To compile without launching, use `./build-linux.sh puzzle`. Its output is `build/puzzle/last_portal`. Missing X11/OpenGL development libraries must be installed through your Linux distribution or lab setup; a C syntax repair will not fix a missing system library.

raylib headers/libraries, fonts and game assets are supplied. Python, CMake and a game editor are not needed. Both build scripts stop on compilation failure. Reopening the old executable alone does not apply a repair: save and repeat Build and Run after every source change.

Building and running for debugging

For a build with source line information and saved compiler/runtime messages, use the supplied **debug launchers**. Save your four student C files and close the running Puzzle game first. These launchers rebuild your Puzzle code before opening it. They require the same compiler and system libraries as the ordinary build.

One launcher from the outer pack folder

Windows: double-click `DEBUG_PUZZLE_WINDOWS.bat`. It builds and runs your Puzzle, then waits for a key before closing so that error messages remain visible.

Linux: open a terminal in the outer extracted pack folder and run:

```
sh debug-puzzle-linux.sh
```

Both launchers switch into `Puzzle` automatically. You can also open a terminal directly in `Puzzle` and use these commands:

Action	Windows PowerShell in Puzzle	Linux terminal in Puzzle
Build and play	.\debug-windows.bat	sh debug-linux.sh
Build only	.\debug-windows.bat build-only	sh debug-linux.sh --build-only
Run under GDB	.\debug-windows.bat gdb	sh debug-linux.sh --gdb

Where to find the messages

- `Puzzle/build/puzzle-debug/build.log` records the compiler output. If compilation fails, the script prints it and does not open a game. Correct the first error, save and run the launcher again.
- `Puzzle/build/puzzle-debug/run.log` records game output and errors in a default run. The script prints it after the game closes or crashes. A new run replaces this log.
- The separate debug executable is `Puzzle/build/puzzle-debug/last_portal.exe` on Windows or `Puzzle/build/puzzle-debug/last_portal` on Linux. It uses your Puzzle source and `puzzle.save`, and keeps the Reference game separate.

The build enables `-g3 -O0 -fno-omit-frame-pointer` alongside the normal warnings. This retains debug information and avoids optimisation so that a debugger can relate execution to C source lines. An unfinished operation still shows its repair screen; the debug launcher does not implement or bypass it.

Optional tools for a crash

GDB must be installed separately for the GDB mode. When a crash stops the game, type `bt` to show the call stack, `frame 0` to select the failing frame, and `list` or `print variable_name` to inspect it. Type `quit` to leave. GDB writes its session to `build/puzzle-debug/gdb.log` instead of the default run log.

On Linux, `sh debug-linux.sh --sanitize` from `Puzzle` adds AddressSanitizer and UndefinedBehaviorSanitizer. If supported by your compiler, they report detected invalid memory access or undefined behavior in `run.log`, often with a source line. Use the default mode if these tools are unavailable.

See `Puzzle/DEBUGGING.md` for the same commands and extra guidance. Debugging tools help locate faults; you still need to verify the stack, queue, list and controller contracts in this handout.

How the supplied project is organised

The game separates data structures, game rules and presentation. Your functions manipulate the data they receive. The engine decides when to call them and what a successful operation means on screen.

File or folder	Role and student action
`puzzle/student/stack.c`	Implement `history_push` and `history_pop`. Keep the supplied initializer.
`puzzle/student/queue.c`	Implement `queue_enqueue` and `queue_dequeue`. Keep initialization and peek behavior.
`puzzle/student/attack_list.c`	Implement insertion and removal. Finding, copying IDs and freeing the whole list are supplied.
`puzzle/student/diagnostics.c`	Diagnose and repair `shield_accepts` and `select_finisher_power`. Both already contain compiling code.
`include/structures.h` and `include/diagnostics.h`	Define shared types, constants and function declarations. Read them; preserve their interfaces.
`src/main.c`	Creates the window, handles keyboard/buttons, manages screens, saves room checkpoints and runs the main loop.
`src/game.h` and `src/game.c`	Own the live game state, board rules, snapshots, random paths, courier states, duel states and win conditions.
`src/view.c`, `src/view.h`, `src/memory_view.h`, `src/duel_view.h`	Draw the board, characters, route reveal, queue, attacks, effects, controls and overlays. Student rendering code is not required.
`src/repair.c`, `include/repair.h`	Provide task messages, behavior checks and the repair marker. Do not bypass or alter these checks.
`src/story.h`, `assets/`, `vendor/`	Provide chapter text, art, fonts, audio resources and bundled libraries.
Build scripts and `tests/check_repair.c`	Compile the chosen edition and provide the optional text-only repair check.

One action through the system

```
Player presses a key or clicks a button
    -> main.c selects a game action
    -> game.c calls a data-structure operation
    -> your operation returns a value or changes its structure
    -> game.c updates the world if the operation succeeded
    -> view.c draws the resulting state
```

For example, an arrow in mission two is converted into a `Command`. The engine passes that value to `queue_enqueue`. The bot stays still while you plan. After launch, the engine calls `queue_dequeue`, obtains the oldest Command and then moves the bot. Neither student queue function reads keys or draws a sprite.

What the remaining code already does

The engine calls your repaired data operations during play. The functions below are supplied; keep your changes in the four student files.

Supplied part	Responsibility
Setup and state	`game_init`, `game_load_room` and `game_destroy` prepare/reset the world and release nodes. `Game` owns live positions, queues, history, courier/duel states and flags.
Movement and undo	`game_move` enforces board rules and exits. Snapshot/remember/restore helpers capture values and rebuild attack nodes. `game_undo` asks your history operation for a saved board.
Memory reveal and planning	`game_memory_reveal` starts the 30-second preview; `game_memory_hide` ends it. Clear/Backspace actions edit the planned queue.
Courier execution	`game_run_bot` requires a complete plan. `game_step_bot` uses your dequeue, checks the exact next path step and advances, delivers or destroys the bot.
Duel sequence handling	`game_duel_start` begins the encounter. Openings use head insertion; counters use your queue. `game_submit_counters` asks your shield routine to validate the response.
Return fire and finishing	Return shots remove a real node and dequeue a counter. `game_finisher` checks nine blocks and both controllers before the beam. `game_ready` and `game_next_room` control progression.
Timing and display	`game_tick` advances timers and animations. Drawing/story code shows the current state. F1 and pause screens stop gameplay updates; the route is covered while its reveal is paused.

Helpers in the student files and headers

Helper	Already implemented behavior
`history_init`	Sets the saved-state count to zero.
`queue_init` and `queue_peek`	Create the empty queue and inspect its oldest entry without removing it.
`attack_contains`	Finds whether an ID already occurs in the list.
`attack_copy_ids` and `attack_free`	Copy IDs for value snapshots, or release every live node and clear the head.
`counter_command` and `counter_symbol`	Convert a closing bracket to its Command value or a shield Command back to its bracket.

Why progress stops at a repair

After the story, the engine checks the operations needed for the current room. Mission one needs both history functions; mission two also needs both queue functions; mission three also needs both list functions. Earlier repairs are checked again when resuming a later room. Shield diagnosis occurs when a response is submitted, and charge diagnosis occurs when the finishing move is attempted.

Data and ownership

A **value copy** duplicates data. A **pointer** refers to an existing object. This distinction matters: history owns stored values, whereas the attack list owns dynamically allocated nodes.

Core structures

```
typedef struct { int x, y; } Cell;

typedef struct {
    Command items[COMMAND_CAPACITY];
    int front, rear, count;
} CommandQueue;

typedef struct {
    BoardState items[HISTORY_CAPACITY];
    int count;
} History;

typedef struct AttackNode {
    int id;
    char opening;
    struct AttackNode *next;
} AttackNode;
```

Constant	Value and meaning
`HISTORY_CAPACITY`	128 complete saved BoardState values.
`COMMAND_CAPACITY`	64 usable Command cells. Use the constant, not a four-move assumption.
`MEMORY_PATH_MAX`	14 moves; the path array has one extra Cell for its start.
`MEMORY_REVEAL_SECONDS`	30 seconds; Start planning now can hide the path sooner.
`DUEL_SEQUENCE_MAX` and `DUEL_WAVES`	At most 4 openings in one sequence; 3 sequences in the duel.
`DUEL_ATTACK_TOTAL` and `ATTACK_CAPACITY`	9 matched pairs to finish; snapshots have room for 8 pending attack entries.

What a complete BoardState contains

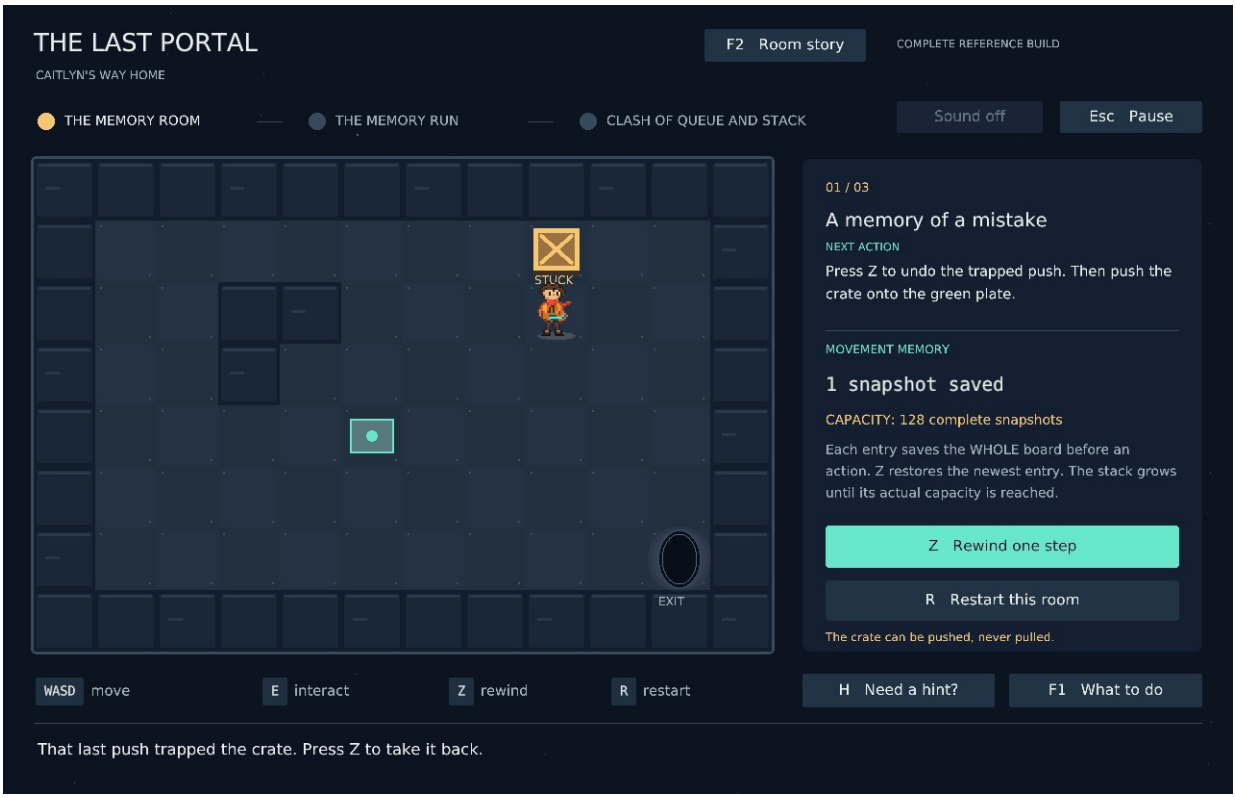
A snapshot includes `hero`, `crate` and `bot`; both `queue` and `counter_queue`; `attack_ids`, `attack_openings` and `attack_count`; `delivered`; the complete `DuelState`; and the complete `CourierState`. Copy the entire supplied value. Attack snapshots store IDs/brackets rather than pointers, so changing or freeing a live node cannot damage an older saved state.

`CourierState` stores `path`, route `length`, current `step`, `attempt`, `phase` and timer `clock`. `DuelState` stores phase, wave, sent/block counts, mistakes, the last attack/counter, clock and result flags. These are engine-managed records, not new student implementation tasks.

A valid queue has `count` between 0 and 64. Empty means `front = rear = -1`. Otherwise front is the oldest pending entry and rear is the last inserted entry; both are valid array indices. Unused array cells are not pending Commands.

Mission 1 Restore a memory of a mistake

Caitlyn has pushed a crate against the upper wall. It can be pushed but cannot be pulled. The game has prepared the board state from before that mistake, but restoring it requires your history stack.



Mission one begins with a trapped crate. The panel shows the true saved count and the capacity of 128 whole snapshots.

Play the repaired room

1. Implement both history operations in `puzzle/student/stack.c`, close the game, save and rebuild.

2. Press Z to restore the board before the trapped push.

3. Walk beside the crate using WASD/arrows and push it onto the green pressure plate. H provides a gameplay route if needed.

4. When the plate powers the doorway, walk into EXIT at the lower right. Press Enter to continue. R restarts this room.

What the engine passes to your stack

Before a board action, the engine creates a `BoardState` and passes its address to `history_push`. Later, `history_pop` writes one complete saved value into the caller's `out` destination. The engine restores positions and other state from that returned value. Your functions do not decide when to save, move Caitlyn, or render the old board.

One history entry is **one complete board before an action**. It is not one direction, one key press or a pointer to the current world. The display is a count of complete snapshots, not a four-move stack.

Repairs 1 and 2 History stack

Edit only the two missing bodies in `puzzle/student/stack.c`. The structure and initializer are supplied.

```
bool history_push(History *history,
                  const BoardState *state);
bool history_pop(History *history, BoardState *out);
```

Repair 1 Save a state

- On success, store a complete independent value copy of `*state` at the stack top, increase `count` exactly once and return true.
- A NULL history pointer, NULL state pointer or full history must return false without changing the stored stack.
- Later changes to the caller's live state must not alter any saved snapshot. Do not allocate a separate node or save the pointer passed as `state`.

Repair 2 Restore the latest saved value

- On success, copy the newest complete saved state into `*out`, remove that occupied entry, decrease `count` exactly once and return true.
- If the history is empty, or either argument is NULL, return false. Leave both the stack and the caller's output unchanged on rejection.
- LIFO means last in, first out: save A, then B; the first pop returns B, the next returns A. Do not remove the oldest state.

Required behavior trace

Action	Stored states from oldest to newest	Result
Initialise	empty	count 0
Save A	A	count 1
Save B	A B	count 2; changing live B leaves saved B intact
Pop into out	A	out is B; count 1
Pop into out	empty	out is A; count 0
Pop while empty	empty	false; out remains unchanged

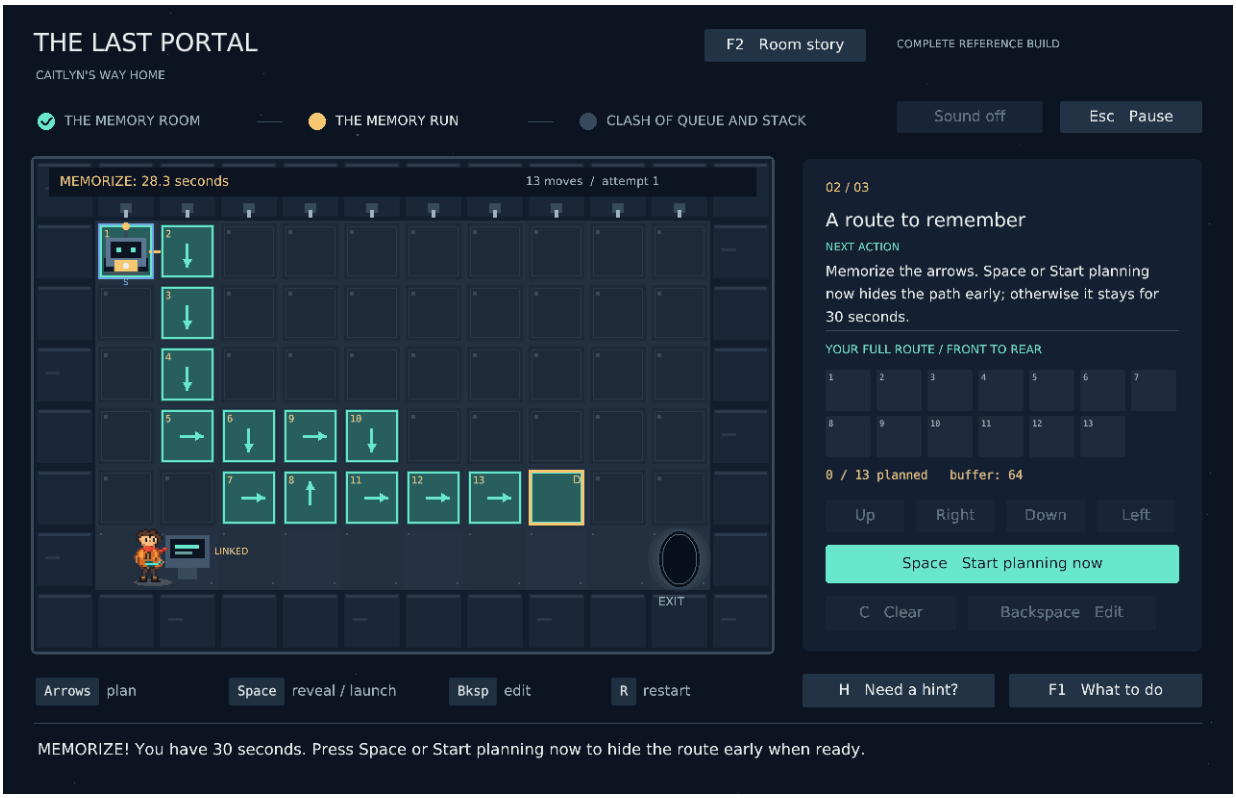
For the opening mistake, the earlier board has Caitlyn at `(8,3)` and the crate at `(8,2)`. The later trapped board has positions `(8,2)` and `(8,1)`. The saved state must still contain the earlier positions after the live board changes.

Self checks

Check empty and one-entry histories; several pushes/pops; all 128 usable entries; rejection of the next push; NULL arguments; independence of saved values; and copying fields beyond hero coordinates, including both queues and courier/duel state. Each operation should use a constant number of stack operations and copy one fixed-size BoardState.

Mission 2 A route to remember

The courier carries a power cell across a security floor. A random route has 10 to 14 moves. You must remember it and enter the entire plan before launching the bot.



The path is revealed for 30 seconds. Start planning now hides it early; the bot waits while the complete remembered plan is entered.

Play the memory challenge

1. Repair both queue operations, rebuild, then press Space to reveal the random route. Follow the numbered arrows from S to D.

2. Study it for up to 30 seconds. Click Start planning now, or press Space/Enter again, when ready to hide it early. F1/Esc pause and cover the route.

3. After it disappears, queue ALL directions with arrows/WASD or the buttons. The side panel shows your whole plan. Backspace edits the last arrow; C clears.

4. Space/Enter launches only when every move is entered. The bot dequeues and executes the plan. One wrong next step triggers the laser and destroys it.

5. R creates a different route for another attempt. No single-step probing, mid-run edits or rewind can rescue an attempt. H gives strategy, never the answer.

6. A perfect run delivers the cell and releases Caitlyn. Walk right into EXIT to reach the duel.

Planning has no response deadline after the reveal. The engine supplies random generation, timing, safety comparison and laser animation. Your queue operations only store and retrieve Command values.

Repairs 3 and 4 Circular command queue

Edit `puzzle/student/queue.c`. The same queue implementation serves both the courier and Caitlyn's countersequence in the final duel.

```
bool queue_enqueue(CommandQueue *queue, Command command);
bool queue_dequeue(CommandQueue *queue, Command *out);
```

What the queue stores

Each entry is one `Command` enum value, not a coordinate or a keyboard character. The movement values are `CMD_UP`, `CMD_RIGHT`, `CMD_DOWN`, `CMD_LEFT`. The shield values are `CMD_BLOCK_ROUND` for `)`, `CMD_BLOCK_SQUARE` for `]`, and `CMD_BLOCK_CURLY` for `}`. Enqueue must accept all seven documented values and reject an invalid enum value.

Repair 3 Record a Command at the rear

- On success, record the new Command after the existing pending Commands, update rear/count and return true. The first insertion into an initialised empty queue occupies slot 0 and sets both indices to 0.
- Use all `COMMAND_CAPACITY` cells. The index after the final array cell wraps to 0. Keep front pointing to the oldest pending entry.
- A NULL queue, invalid Command or full queue returns false without changing the queue. Do not hard-code a four-entry buffer or reserve an unused cell.

Repair 4 Retrieve the oldest Command

- On success, write the oldest pending Command into `*out`, remove that pending entry, advance front circularly, decrease count and return true. Preserve FIFO order across wraparound.
- After the final removal, restore the canonical empty state: count 0 and front/rear both -1.
- An empty queue or NULL argument returns false without changing the queue or output. Dequeue does not move the bot; the engine does that after success.

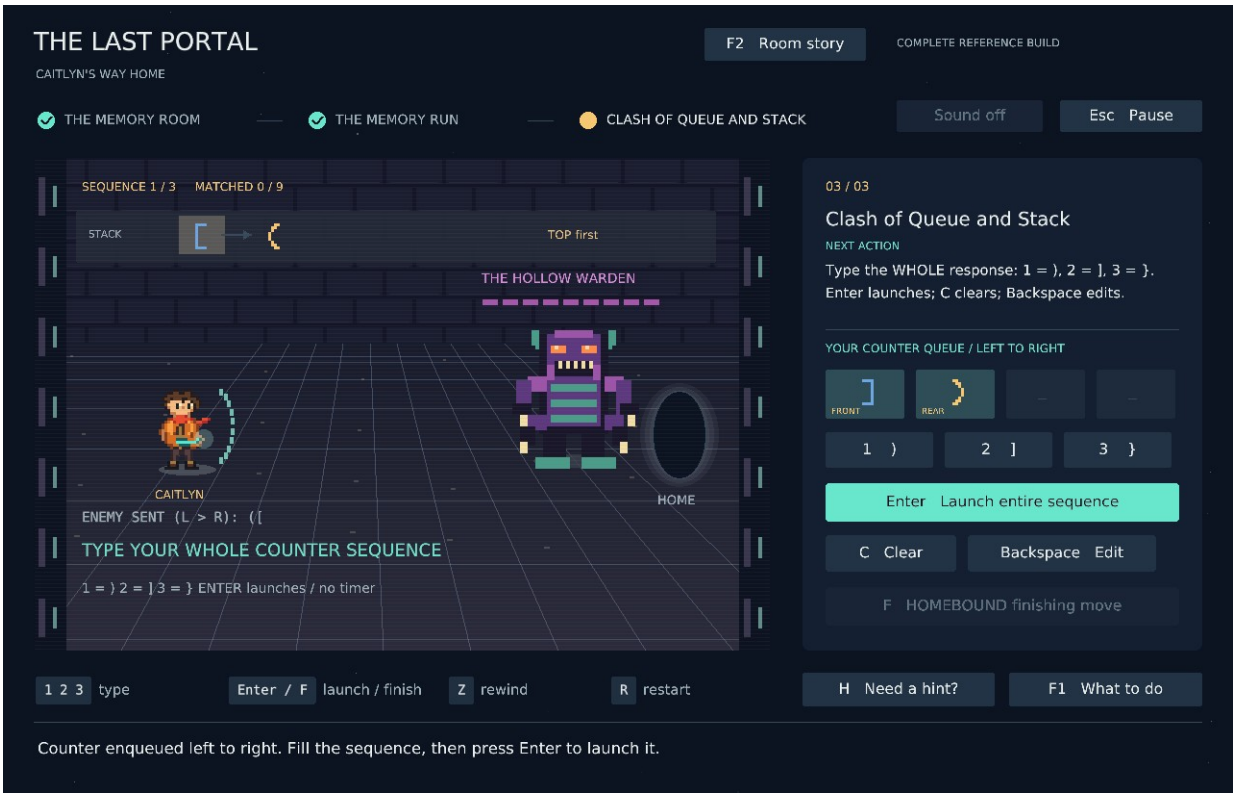
A wraparound example using the actual capacity

Before	Operation	After
front 62, rear 63, count 2 slot 62 RIGHT; slot 63 UP	Enqueue DOWN	front 62, rear 0, count 3 slot 0 DOWN
RIGHT, UP, DOWN pending	Dequeue	out RIGHT; front 63, rear 0, count 2
UP, DOWN pending	Dequeue twice	out UP, then DOWN; empty -1/-1

The courier's whole route fits in 64 cells. During a run, the next supplied direction must match the **exact next route step**. Reversing FIFO order or losing a wraparound entry can therefore cause a real wrong-step failure. Test FIFO, all 64 cells, wraparound, empty/full rejection and shield Commands separately from testing your memory.

Mission 3 Clash of Queue and Stack

The Hollow Warden blocks the only alley leading to Home. The enemy sends complete sequences of opening brackets. Caitlyn must queue the corresponding complete closing sequence and send the attacks back.



The top strip shows the enemy linked stack, with TOP first. The side panel shows the player counter queue, with FRONT first.

Play the duel

1. After repairing list insertion/removal, press Space to begin. The engine reads each incoming opening sequence left to right and pushes each opening at the head of the linked list.

2. Type the whole closing response left to right: 1 means), 2 means], and 3 means }. Buttons or the closing brackets themselves also work. C clears; Backspace edits.

3. Enter submits the whole response. The shield compares enemy TOP against queue FRONT. A valid response triggers return shots automatically; each matched pair removes one node and dequeues one counter.

4. Clear all three sequences of lengths 2, 3 and 4. After all nine pairs, press F to attempt HOMEBOUND. Repair the finishing controller when its diagnosis appears.

5. When the beam defeats the Warden, hold Right/D to follow the cleared alley into Home.

For example, the enemy sequence `([` needs response `])`. The first comparison is `[` with `]`, then `(` with `)`. A wrong response preserves both structures for editing and retry. There is no reaction timer. The shield logic diagnosis appears when a response is first submitted; its repair is detailed later in this handout.

Repairs 5 and 6 Linked attack nodes

Edit `puzzle/student/attack_list.c`. Each node has a unique integer `id`, an `opening` payload and `next`. A NULL head is an empty list; a tail has `next == NULL`. The pointer-to-pointer `AttackNode **head` lets your function change the caller's first-node pointer.

```
bool attack_insert_after(AttackNode **head,
                         int predecessor, int id,
                         char opening);
bool attack_remove(AttackNode **head, int id);
```

Repair 5 Allocate and insert

- Allocate a node and copy both the ID and opening payload. The opening must be `(`, `[` or `{`.
- If predecessor is -1, insert before the current head, including into an empty list. Otherwise find the node with that predecessor ID and insert immediately after it. Preserve its former successor.
- IDs identify nodes; they are not array positions or list indices. Your function must support other unique IDs used in checks, not just the particular attacks seen during play.
- Reject a NULL head argument, duplicate ID, invalid opening, missing predecessor or allocation failure. Return false with the original list unchanged and no leaked allocation. Return true on success.

Repair 6 Unlink and release

- Find the requested ID, reconnect the surrounding links, update the caller's head when necessary, free exactly the removed node and return true.
- Head, middle, tail and final-node removal must work. A missing ID or NULL head argument returns false unchanged. Do not free a neighbour or leave a link to released memory.

Required list behavior

Initial list	Request	Result
empty	Insert ID 10 with predecessor -1	10
10	Insert ID 30 after ID 10	10 -> 30
10 -> 30	Insert ID 20 after ID 10	10 -> 20 -> 30
10 -> 20 -> 30	Remove ID 10	20 -> 30; head changes
20 -> 30	Insert a duplicate ID 20	false; same list
20 -> 30	Remove missing ID 99	false; same list

The duel uses this general list as a stack: each opening is inserted at the head, so the final arriving opening becomes TOP. Each accepted counter removes that TOP by its ID. General middle/tail operations are still part of the contract even though this gameplay primarily uses the head. Insertion owns its new allocation; removal and the supplied cleanup helper release nodes.

Repair 7 Diagnose the shield controller

In `puzzle/student/diagnostics.c`, the shield routine already compiles. Its equal-length calibration can accept a wrong response. Diagnose the existing logic and implement the complete rule below. There is no missing-function marker in this file.

```
bool shield_accepts(const AttackNode *enemy_top,
                    const CommandQueue *counter_queue);
```

Inputs and expected result

`enemy_top` is the head of the existing linked stack of openings. `counter_queue` is a circular queue of closing Commands, read from its current front for exactly count entries. Both sequences were originally read **left to right**. The incoming sequence was pushed, and the player response was enqueued.

Return true only when the stack and queue have equal lengths and every paired type matches in the required order: `(` with `)`, `[` with `]`, and `{` with `}`. Compare stack TOP with queue FRONT. Both inputs are const: validation must not consume, relink, dequeue, free or otherwise modify either structure. The engine performs the actual removals only after acceptance.

Trace the order before writing code

Step	Enemy stack from TOP	Player queue from FRONT	Meaning
Read enemy (	(	empty	First opening is pushed.
Read enemy [	[-> (	empty	Newest opening becomes TOP.
Type] then)	[-> (	] ->)	Typed order is preserved by the queue.
First comparison	[	]	Types match.
Next comparison	(	)	Types match; both finish together.

Thus enemy `([` with response `])` is valid; response `)]` has the right length but wrong order/types. This API receives a linked stack and a queue, not a single expression string. Keep the supplied signature.

Rejection cases and boundaries

- An empty enemy stack and an empty, valid queue are accepted. The empty queue must have count 0 and front/rear -1.
- A NULL queue pointer, unequal lengths, invalid opening, movement Command used as a shield counter, mismatched type or invalid queue state must be rejected.
- For a nonempty queue, count is within capacity and both indices are valid. Rear must equal `(front + count - 1) % COMMAND_CAPACITY`. Entries must be read circularly, not by assuming front is 0.

Test correct pairs, wrong equal-length pairs, missing/extra counters, empty inputs, invalid data and wraparound. Also check that the list links, payloads, queue contents and indices are identical before and after validation. H on the diagnosis panel offers an optional reasoning clue; it does not apply the repair.

Repair 8 Diagnose the finishing charge

After all nine pairs are blocked, HOMEBOUND still needs a safe charge. The Warden jams the highest power band. The controller must choose the highest **distinct** value below that maximum. The existing code in `puzzle/student/diagnostics.c` can count duplicate maxima as separate ranks.

```
bool select_finisher_power(const int *readings,
                           int count, int *out);
```

Required behavior

- On success, write the second-highest distinct integer into `*out` and return true. Repeated equal readings describe the same band.
- Leave the input readings unchanged. Values can be negative, zero or any representable int, including `INT_MIN` and `INT_MAX`.
- If readings or out is NULL, count is less than 2, or fewer than two distinct values exist, return false. A rejected call must leave the caller's output value unchanged.
- Solve the general problem. Hard-coding the monitor's answer or depending on one ordering of readings does not meet the contract.

Examples that the repaired controller must distinguish

Input readings	Expected result
88, 88, 57, 31, 57	true; output 57
4, 9, 1, 6, 9	true; output 6
-8, -3, -3, -6, -9	true; output -6
0, INT_MIN	true; output INT_MIN
INT_MAX, INT_MIN, INT_MAX	true; output INT_MIN
7, 7, 7	false; leave output unchanged
One reading or no readings	false; leave output unchanged

If the output variable begins with a test value such as 123, a failed call for `[7,7,7]` must still leave 123 there. Writing a made-up reserve and returning false is also incorrect. Do not treat a numerical sentinel such as -1 as proof that a band exists: -1 can be an ordinary reading.

Expected reasoning and gameplay effect

Choose an algorithm that maintains the distinction between a repeated peak and an actual lower band. Aim for linear time and constant extra storage, and be able to explain the complexity of your implementation. You may use different variable names and internal logic while preserving the interface and complete contract.

The game uses this selection in its real finishing interlock. F remains unsuccessful if the controller is unsafe, even after the attacks are blocked. Once the shield, charge and previous repairs work, F starts the beam, the alien falls and the route Home opens. Rebuilding during diagnosis restarts the current room, so you play the duel again.

Checking and debugging your repairs

The game is the main demonstration. A supplied text-only checker can also compile your student code and report the first unfinished or incorrect task. Run it from the extracted project folder.

Platform	All tasks	Earlier room checks
Linux	`./check-linux.sh`	`./check-linux.sh 0` `./check-linux.sh 1`
Windows PowerShell	`.\check-windows.bat`	`.\check-windows.bat 0` `.\check-windows.bat 1`

The command-line room numbers are 0, 1 and 2 for game missions 1, 2 and 3. The default is 2, which checks all foundations and both diagnoses. A nonzero exit is expected while a repair remains. The game can still let you play an earlier room once that room's required functions pass.

Minimum self checks

- History: complete independent values; LIFO; 128 usable entries; empty/full/NULL rejection; output unchanged on failure.
- Queue: FIFO; all 64 cells; wraparound; empty reset to -1/-1; all seven valid Commands; rejection leaves state/output unchanged.
- List: empty, head, middle, tail and final-node cases; unique IDs; valid payloads; missing predecessor/ID; correct allocation ownership.
- Shield and charge: correct and incorrect examples, empty/invalid inputs, circular queue order, distinctness, negative/extreme readings and no input mutation.

Common symptoms

Symptom	What to check
The same repair appears after editing.	Did you save and rebuild the Puzzle edition from the correct extracted folder? Did compilation succeed? Is the marker still in that body?
Compilation stops on a warning.	The supplied scripts treat warnings as errors. Read the first message and fix its cause. Keep declarations and return types consistent.
The game crashes or the list behaves unpredictably.	Trace NULL pointers, allocation size, head updates, lost successors, invalid indices and use-after-free. Only the removed node should be released.
The bot is destroyed.	First check the remembered route and the exact next step. A gameplay mistake is possible even with correct queue code. Separately test FIFO and wraparound.
The duel rejects a response of the right length.	Check bracket types and TOP versus FRONT order. Counts alone are insufficient. Do not consume inputs inside validation.
A later room still reports a basic repair.	Earlier functions are deliberately checked again. Passing one example is not enough; keep every previous operation correct.

The checks are representative, not a proof that arbitrary student C code is safe. A bad pointer can crash the program and an infinite loop can hang it. Trace small cases first. Do not change the engine, tests, constants, game paths or validation rules to bypass a failure.

Completion and submission

What counts as complete

All six foundation operations and both controllers must satisfy their contracts, and you must be able to explain your own code and its boundary cases. Rebuild from your submitted source, complete the three rooms and reach **HOME AT LAST**. The final screenshot alone is not a substitute for functioning C implementations.

Submit one ZIP

Keep the source paths below inside your submission. Use the student identification and naming rules given in your course portal, and upload using that portal's assignment link and deadline.

```
YourSubmission/
    puzzle/student/stack.c
    puzzle/student/queue.c
    puzzle/student/attack_list.c
    puzzle/student/diagnostics.c
    README.txt
    evidence/home.png
```

In `README.txt`, include your name/roll number, platform and build command. In about 150 to 200 words, explain the cause of each controller fault, what you changed, and one boundary case you used to check each repair. Include one screenshot of your successful ending as `evidence/home.png`. A formal report, certificate, slide deck, new artwork and video are not required.

Submit the source, not only an executable or a save file. Keep the supplied interfaces unchanged. Only the four student C files are implementation deliverables; the game engine, assets and libraries are already supplied. A fresh copy of the Puzzle project with your four files must compile and run with the standard scripts.

Controls reference

Context	Controls
Story	Enter/Esc begins or returns to the room; Space pauses the crawl; R replays; F1 opens the guide.
Any active room	F1 guide; F2 story replay; R room restart; H hint; Esc pause; M mute. Guides/pause stop gameplay updates.
Mission 1	WASD/arrows walk and push; Z restores the latest saved board.
Mission 2 reveal	Space reveals; during reveal, Start planning now or Space/Enter hides early. Otherwise it ends after 30 seconds.
Mission 2 planning and run	Arrows/WASD plan; C clears; Backspace edits; Space/Enter launches a full plan. R retries. Z cannot rescue a run or revive the bot.
Mission 3	Space begins; 1/2/3 types) /] / }; Enter submits; C clears; Backspace edits; F finishes after all nine blocks; Right/D walks Home.
Repair or diagnosis screen	F1 explains the room. H toggles a diagnosis clue. Q/Esc closes the current build for editing. Rebuild after saving source changes.

Room checkpoints are stored in `puzzle.save`; individual moves and source repairs are not stored there. Reopening a checkpoint rechecks the code. In the duel, wait for a shot or beam to finish before Z. Completing the code and completing the game are both part of this assignment.